Row Key & Sharding: In Bigtable, data is dynamically partitioned (sharded) across nodes based on the Row Key. What is your proposed Row Key for this dataset? Explain how this choice avoids "hotspotting" and impacts your ability to efficiently query vehicles by *City* or *Make*.

My proposed Row Key for this dataset is postal code#city#make#DOL. Using the postal code as the leading key avoids hotspotting because it distributes the row entries more evenly than if the city or make were the leading key (city is heavily skewed towards Seattle and Make is heavily skewed toward Tesla). This also improves the efficiency of city-based queries with city as the second key, since postal code is also location-dependent. This does come at the cost of increased latency of queries which scan vehicles based on Make, since they are not all grouped together.

Column Families: How would you design the Column Families for this dataset? Group the attributes logically and explain how this aids in distributed read performance.

I would group the columns into two families - location and vehicle_info, grouping the following columns:

- ***location:*** *State, County, City, Postal Code, Legislative District, Vehicle Location, Electric Utility, 2020 GeoID*
- ***vehicle_info:*** *VIN, Model Year, Make, Model, Electric Vehicle Type, CAFV Eligibility, Electric Range, DOL Vehicle ID*

Grouping the columns in this way allows distributed reads to perform more efficiently because it allows BigTable applications to retrieve only the column families required for a particular query while avoiding unnecessary retrieval. Say an application only needs to query statistical data on Electric Range and Make - the application can do this without over-retrieving data from the location column family.

Horizontal Scaling: Compare the pros and cons of using a *Hashed Shard Key* (e.g., on `VIN`) versus a *Range-Based Shard Key* (e.g., on `Model Year`) regarding query routing efficiency and cluster performance.

A hashed shard key such as one using the VIN helps further avoid hotspotting since it distributes each row more evenly across the shards, at the cost of less efficient scans based on model year or Make or some other attribute. However, a range-based shard key such as on model year shards entries with the same model year on contiguous key-ranges, allowing better performance for scans based on model year at the risk of hotspotting.

Performance Analysis

The measured average latency for /insert-fast was 92.73ms while the measured average latency for /insert-safe was 92.88 ms. Although the measured difference may seem small, at much larger throughputs the availability and latency gap becomes much more apparent than in

our limited test of 500 of each post type. Choosing a primary-only write acknowledgement instead of a majority acknowledgement reduces the time required to confirm writes because the database does not need to wait for replication to a majority of replica nodes; this comes at the cost of potentially weakened durability if the primary fails before the data is replicated. One such use case for primary-only write acknowledgement could be continuous vehicle telemetry collection. For noncritical telemetry updates, occasional data loss may be an acceptable tradeoff in exchange for much higher processing throughput at lower latency to handle the incoming data.